**МИНИСТЕРСТВО НАУКИ, ВЫСШЕГО ОБРАЗОВАНИЯ И ИННОВАЦИЙ КЫРГЫЗСКОЙ РЕСПУБЛИКИ**

**Кыргызский государственный технический университет им. И. Раззакова**

Институт информационных технологий

Кафедра «Обеспечения безопасности информационных систем»

# ЛАБОРАТОРНАЯ РАБОТА № 2

**по дисциплине «Структуры данных и алгоритмы»**

Тема: «Условные конструкции, циклы и функции в Python»

Выполнил: Памиров Казбек Алмазбекович

Группа: Бис-1-26

Проверила: Сарыпбекова Ж.Р.

Бишкек — 2026

## 1. Цель работы

Изучить и закрепить практические навыки применения условных конструкций `if`, `elif`, `else`, циклов `for` и `while`, функции `range()`, операторов управления циклом `break` и `continue`, а также создания функций с помощью `def` и возврата результата через `return`.

## 2. Краткие теоретические сведения

Конструкция `if/elif/else` выбирает одно действие из нескольких: условия проверяются сверху вниз, выполняется первая ветка с истинным условием, а `else` срабатывает, когда все условия ложны. Цикл `for` применяют, когда число повторений известно заранее или нужно перебрать последовательность; функция `range(start, stop, step)` выдаёт числа от `start` до `stop - 1`, то есть конечное значение в диапазон не входит.

Цикл `while` повторяет тело, пока условие истинно, поэтому внутри цикла должны изменяться переменные, от которых зависит условие, иначе цикл станет бесконечным. Оператор `break` сразу завершает ближайший цикл, `continue` пропускает остаток текущей итерации и переходит к следующей. Функция создаётся через `def`, получает данные через параметры и возвращает результат оператором `return`.

Трассировка — пошаговая запись значений переменных при выполнении алгоритма. Инвариант цикла — утверждение, которое верно до цикла и сохраняется после каждой итерации; для накопления суммы это «`total` равна сумме уже обработанных подходящих чисел». Временная сложность показывает рост числа операций с ростом объёма данных n, пространственная — рост используемой памяти.

## 3. Демонстрационный пример: сумма и количество положительных чисел

### 3.1. Постановка задачи

Пользователь вводит целые числа, ввод заканчивается числом 0. Найти сумму и количество положительных чисел, отрицательные числа пропускать. Если положительных чисел не было, вывести сообщение. Для выхода использовать `break`, для пропуска — `continue`, обработку оформить функцией.

### 3.2. Алгоритм

1) Присвоить `total = 0`, `count = 0`.

2) В бесконечном цикле ввести очередное число `number`.

3) Если `number == 0` — выйти из цикла через `break`.

4) Если `number < 0` — перейти к следующей итерации через `continue`.

5) Иначе прибавить `number` к `total` и увеличить `count` на 1.

6) После цикла вернуть `total` и `count`; если `count == 0`, вывести сообщение, иначе вывести сумму и количество.

## 3.3. Псевдокод и блок-схема

```
НАЧАЛО
  total := 0;  count := 0
  ПОКА ИСТИНА
    ВВОД number
    ЕСЛИ number = 0 ТО выход из цикла          (break)
    ЕСЛИ number < 0 ТО следующая итерация      (continue)
    total := total + number
    count := count + 1
  ВЕРНУТЬ total, count
  ЕСЛИ count = 0 ТО ВЫВОД "Положительные числа не были введены."
  ИНАЧЕ ВЫВОД total, count
КОНЕЦ
```

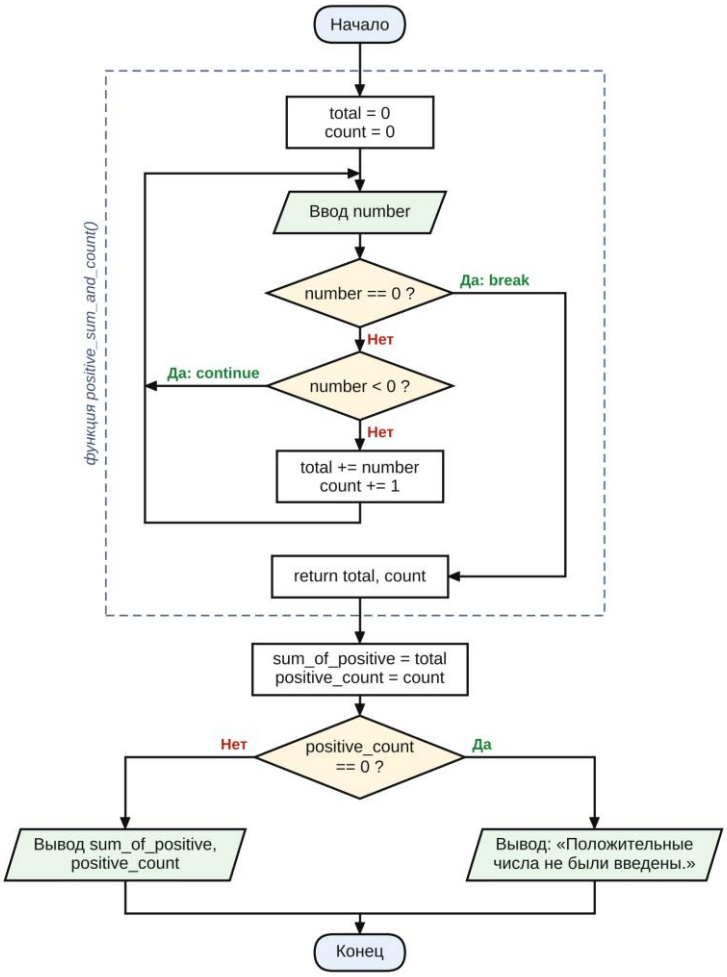



*Рисунок 1 — Блок-схема демонстрационного примера*

### 3.4. Листинг программы

```
def positive_sum_and_count():
    total = 0
    count = 0
    while True:
        number = int(input("Введите целое число (0 — завершить ввод): "))
        if number == 0:
            break
        if number < 0:
            continue
        total += number
        count += 1
    return total, count


sum_of_positive, positive_count = positive_sum_and_count()

if positive_count == 0:
    print("Положительные числа не были введены.")
else:
    print("Сумма положительных чисел:", sum_of_positive)
    print("Количество положительных чисел:", positive_count)
```

### 3.5. Трассировка

| Шаг | number | Действие | total | count |
|---|---|---|---|---|
| 1 | 8 | положительное: прибавить к сумме | 8 | 1 |
| 2 | −3 | отрицательное: continue | 8 | 1 |
| 3 | 5 | положительное: прибавить к сумме | 13 | 2 |
| 4 | −1 | отрицательное: continue | 13 | 2 |
| 5 | 0 | break — выход из цикла | 13 | 2 |

### 3.6. Тестирование

| Ввод | Ожидаемый результат | Фактический результат |
|---|---|---|
| 8, −3, 5, −1, 0 | сумма 13, количество 2 | Сумма положительных чисел: 13<br>Количество положительных чисел: 2 |
| 0 | сообщение об отсутствии положительных | Положительные числа не были введены. |
| −4, −7, 0 | сообщение об отсутствии положительных | Положительные числа не были введены. |

### 3.7. Анализ

Цикл завершается только оператором `break` при вводе 0, поэтому выход определяет переменная `number`. Инвариант: после обработки каждого числа `total` равна сумме всех уже введённых положительных чисел, а `count` — их количеству. Если до нуля введено n чисел, выполняется n + 1 итерация с постоянным числом действий — временная сложность O(n); хранится фиксированный набор переменных — пространственная сложность O(1).

# 4. Задание 1. Ветвление

## 4.1. Постановка задачи

Ввести оценку студента — целое число от 2 до 5 — и вывести её текстовое название: 2 — «Неудовлетворительно», 3 — «Удовлетворительно», 4 — «Хорошо», 5 — «Отлично». При любом другом значении, в том числе при вводе не числа, вывести сообщение об ошибке.

## 4.2. Входные, промежуточные и выходные данные

| Тип | Переменная | Описание |
|---|---|---|
| Входные | `user_input` (str) | строка, введённая пользователем |
| Промежуточные | `grade` (int), `text` (str) | оценка в виде числа; текст, выбранный функцией `grade_to_text` |
| Выходные | `result` (str) | текстовая оценка или сообщение об ошибке |

## 4.3. Алгоритм / псевдокод

```
ФУНКЦИЯ grade_to_text(grade)
  ЕСЛИ grade = 2 ТО text := "Неудовлетворительно"
  ИНАЧЕ ЕСЛИ grade = 3 ТО text := "Удовлетворительно"
  ИНАЧЕ ЕСЛИ grade = 4 ТО text := "Хорошо"
  ИНАЧЕ ЕСЛИ grade = 5 ТО text := "Отлично"
  ИНАЧЕ text := "Ошибка: допустимы оценки от 2 до 5."
  ВЕРНУТЬ text

НАЧАЛО
  ВВОД user_input
  ЕСЛИ user_input — целое число ТО result := grade_to_text(целое(user_input))
  ИНАЧЕ result := "Ошибка: нужно ввести целое число от 2 до 5."
  ВЫВОД result
КОНЕЦ
```

## 4.4. Программа на Python

```
def grade_to_text(grade):
    if grade == 2:
        text = "Неудовлетворительно"
    elif grade == 3:
        text = "Удовлетворительно"
    elif grade == 4:
        text = "Хорошо"
    elif grade == 5:
        text = "Отлично"
    else:
        text = "Ошибка: допустимы оценки от 2 до 5."
    return text


user_input = input("Введите оценку от 2 до 5: ")

try:
    grade = int(user_input)
    result = grade_to_text(grade)
except ValueError:
    result = "Ошибка: нужно ввести целое число от 2 до 5."

print(result)
```

### 4.5. Тестирование

| Ввод | Ожидаемый результат | Фактический результат | Тип теста |
|---|---|---|---|
| 2 | Неудовлетворительно | Неудовлетворительно | корректный |
| 3 | Удовлетворительно | Удовлетворительно | корректный |
| 4 | Хорошо | Хорошо | корректный |
| 5 | Отлично | Отлично | корректный |
| 1 | Ошибка: допустимы оценки от 2 до 5. | Ошибка: допустимы оценки от 2 до 5. | вне диапазона |
| 6 | Ошибка: допустимы оценки от 2 до 5. | Ошибка: допустимы оценки от 2 до 5. | вне диапазона |
| abc | Ошибка: нужно ввести целое число от 2 до 5. | Ошибка: нужно ввести целое число от 2 до 5. | не число |

### 4.6. Блок-схема и анализ




*Рисунок 2 — Блок-схема выбора текстовой оценки*

Выбор выполняется цепочкой `if/elif/else` внутри функции `grade_to_text`: проверки идут по порядку, срабатывает первая истинная, а `else` обрабатывает все остальные числа. Ввод, который нельзя преобразовать в целое число, перехватывается блоком `try/except ValueError`. Циклов нет, выполняется не более четырёх сравнений, поэтому временная и пространственная сложность O(1).

# 5. Задание 2. Цикл for, range() и функция

## 5.1. Постановка задачи

Написать функцию `sum_of_multiples_of_three(limit)`, которая возвращает сумму всех целых чисел от 0 до `limit` включительно, кратных 3. Использовать `for` и `range()`, вернуть ответ через `return`, вызвать функцию для `limit = 50`.

## 5.2. Данные

| Тип | Переменная | Описание |
|---|---|---|
| Входные | `limit` (int) | верхняя граница диапазона, параметр функции |
| Промежуточные | `number` (int), `total` (int) | текущее число цикла; накопленная сумма |
| Выходные | `result` (int) | значение, возвращённое функцией |

## 5.3. Алгоритм / псевдокод

```
ФУНКЦИЯ sum_of_multiples_of_three(limit)
  total := 0
  ДЛЯ number ОТ 0 ДО limit ВКЛЮЧИТЕЛЬНО
    ЕСЛИ number mod 3 = 0 ТО total := total + number
  ВЕРНУТЬ total

НАЧАЛО
  limit := 50
  result := sum_of_multiples_of_three(limit)
  ВЫВОД result
КОНЕЦ
```

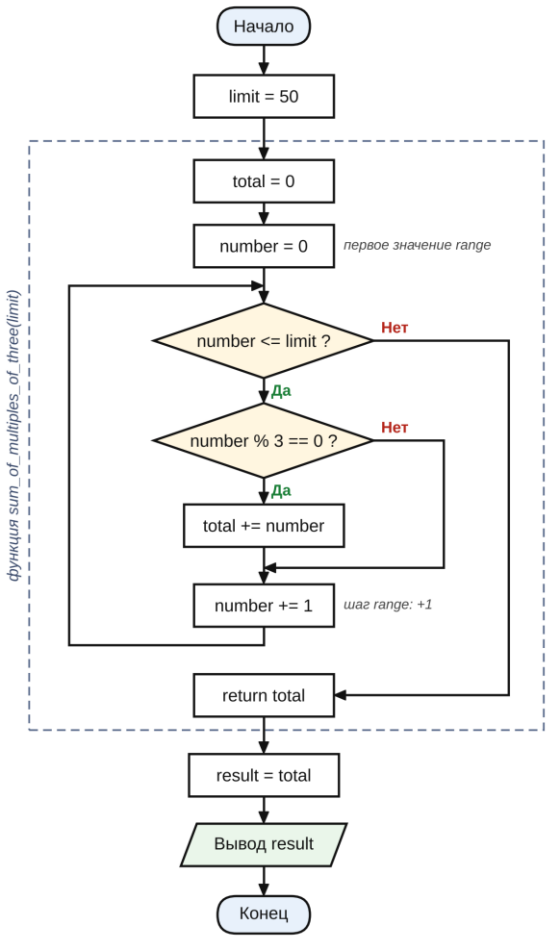



*Рисунок 3 — Блок-схема суммирования чисел, кратных 3*

## 5.4. Программа на Python

```
def sum_of_multiples_of_three(limit):
    total = 0
    for number in range(0, limit + 1):
        if number % 3 == 0:
            total += number
    return total


limit = 50
result = sum_of_multiples_of_three(limit)
print(f"Сумма целых чисел от 0 до {limit}, кратных 3: {result}")
```

## 5.5. Тестирование и результат

| Вызов | Ожидаемый результат | Фактический результат | Пояснение |
|---|---|---|---|
| `sum_of_multiples_of_three(0)` | 0 | 0 | в диапазоне только 0 |
| `sum_of_multiples_of_three(3)` | 3 | 3 | 0 + 3 |
| `sum_of_multiples_of_three(10)` | 18 | 18 | 0 + 3 + 6 + 9 |
| `sum_of_multiples_of_three(50)` | 408 | 408 | 0 + 3 + … + 48 (17 чисел) |
| `sum_of_multiples_of_three(51)` | 459 | 459 | граница 51 кратна 3 и входит в сумму |

Результат запуска программы: «Сумма целых чисел от 0 до 50, кратных 3: 408».

Конец диапазона в `range()` не включается: `range(0, limit)` закончился бы на `limit - 1`. Поэтому используется `range(0, limit + 1)` — он даёт числа 0, 1, …, `limit`, и сама граница тоже проверяется (при `limit = 51` без «+ 1» получилось бы 408 вместо 459).

Анализ: цикл выполняет `limit + 1` итераций с постоянным числом действий — временная сложность O(n), где n = limit. Хранятся только `total`, `number` и `limit`, а `range` не создаёт список в памяти — пространственная сложность O(1). Инвариант: перед очередной итерацией `total` равна сумме уже проверенных чисел, кратных 3.

## 6. Задание 3. Цикл while, break и continue

### 6.1. Постановка задачи

Считывать целые числа до ввода 0 и вывести количество и сумму положительных чисел, а также количество и сумму отрицательных. Завершающий 0 — признак конца ввода, в подсчёт он не входит (поэтому отдельный счётчик нулей не нужен: первый же 0 завершает ввод). Цикл — `while`, выход — `break`, пропуск — `continue`, вычисления — в функции.

### 6.2. Данные

| Тип | Переменные | Описание |
|---|---|---|
| Входные | `number` (int) | очередное введённое число |
| Промежуточные | `positive_count`, `negative_count` | количество положительных и отрицательных чисел |
| Промежуточные | `positive_sum`, `negative_sum` | суммы положительных и отрицательных чисел |
| Выходные | `count_pos`, `count_neg`, `sum_pos`, `sum_neg` | четыре результата, возвращённые функцией |

### 6.3. Алгоритм / псевдокод

```
ФУНКЦИЯ analyze_numbers()
  positive_count := 0;  negative_count := 0
  positive_sum := 0;    negative_sum := 0
  ПОКА ИСТИНА
    ВВОД number
    ЕСЛИ number = 0 ТО выход из цикла                    (break)
    ЕСЛИ number < 0 ТО
      negative_count := negative_count + 1
      negative_sum := negative_sum + number
      следующая итерация                                 (continue)
    positive_count := positive_count + 1
    positive_sum := positive_sum + number
  ВЕРНУТЬ четыре значения
ВЫВОД результатов
```

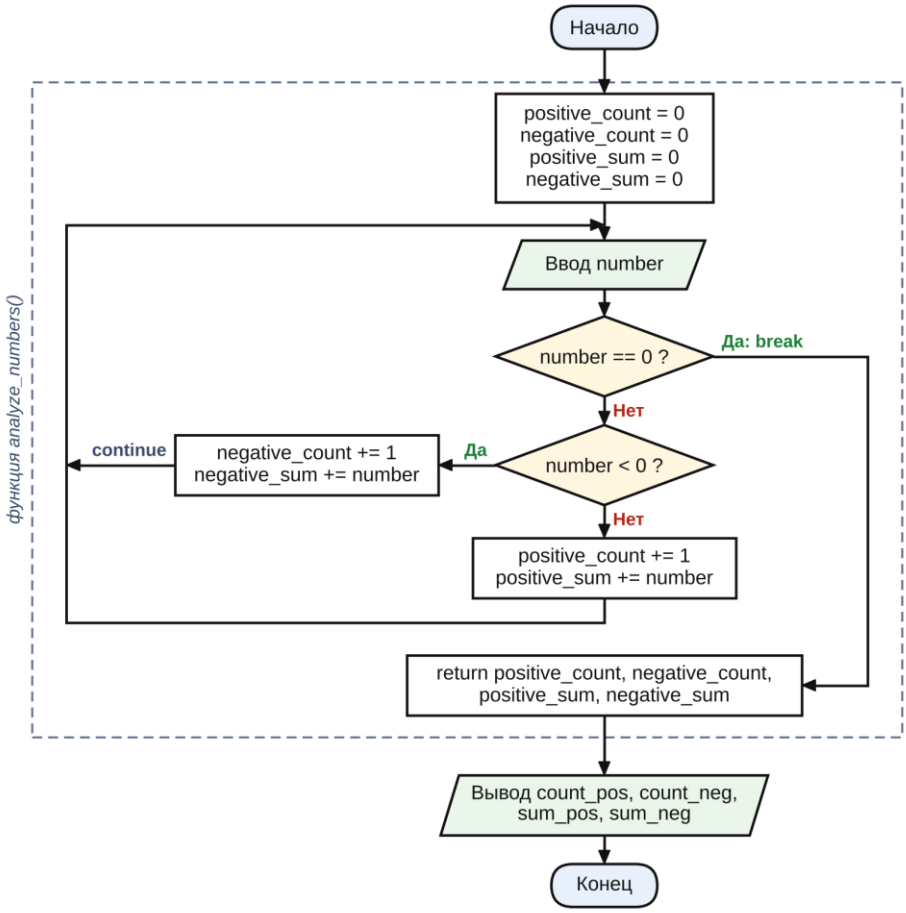



*Рисунок 4 — Блок-схема анализа положительных и отрицательных чисел*

## 6.4. Программа на Python

```
def analyze_numbers():
    positive_count = 0
    negative_count = 0
    positive_sum = 0
    negative_sum = 0
    while True:
        number = int(input("Введите целое число (0 — завершить ввод): "))
        if number == 0:
            break
        if number < 0:
            negative_count += 1
            negative_sum += number
            continue
        positive_count += 1
        positive_sum += number
    return positive_count, negative_count, positive_sum, negative_sum


count_pos, count_neg, sum_pos, sum_neg = analyze_numbers()
print("Количество положительных чисел:", count_pos)
print("Количество отрицательных чисел:", count_neg)
print("Сумма положительных чисел:", sum_pos)
print("Сумма отрицательных чисел:", sum_neg)
```

### 6.5. Трассировка

Ввод: 7, −2, 4, −5, 10, 0. Значения указаны после выполнения шага.

| Шаг | number | Действие | Кол-во + | Кол-во − | Сумма + | Сумма − |
|---|---|---|---|---|---|---|
| 1 | 7 | положительное | 1 | 0 | 7 | 0 |
| 2 | −2 | отрицательное, continue | 1 | 1 | 7 | −2 |
| 3 | 4 | положительное | 2 | 1 | 11 | −2 |
| 4 | −5 | отрицательное, continue | 2 | 2 | 11 | −7 |
| 5 | 10 | положительное | 3 | 2 | 21 | −7 |
| 6 | 0 | break — выход из цикла | 3 | 2 | 21 | −7 |

### 6.6. Тестирование

Формат результата: кол-во положительных / кол-во отрицательных / сумма положительных / сумма отрицательных.

| Ввод | Ожидаемый результат | Фактический результат |
|---|---|---|
| 7, −2, 4, −5, 10, 0 | 3 / 2 / 21 / −7 | 3 / 2 / 21 / -7 |
| 0 | 0 / 0 / 0 / 0 | 0 / 0 / 0 / 0 |
| −1, −2, −3, 0 | 0 / 3 / 0 / −6 | 0 / 3 / 0 / -6 |

### 6.7. Анализ

Цикл `while True` завершается оператором `break`, когда пользователь вводит 0, — выход определяет переменная `number`. В цикле меняются `number` и одна из пар переменных: для отрицательного числа — `negative_count` и `negative_sum`, после чего `continue` не даёт ему попасть в положительные; для положительного — `positive_count` и `positive_sum`. Инвариант: после каждой итерации счётчики и суммы соответствуют всем уже введённым ненулевым числам. Для n чисел временная сложность O(n), память O(1).

## 7. Задание 4. Обработка строк журнала

### 7.1. Постановка задачи

Дан список строк журнала `log_lines`, каждая непустая строка имеет вид «тип_события;IP-адрес». Создать функцию `count_security_events(log_lines)`, которая пропускает пустые строки через `continue`, прекращает обработку на строке `STOP` через `break`, считает события каждого типа и возвращает словарь; результат вывести.

### 7.2. Данные

| Тип | Переменная | Описание |
|---|---|---|
| Входные | `log_lines` (list[str]) | список строк журнала |
| Промежуточные | `line`, `clean_line`, `event_type` (str) | текущая строка, она же без пробелов по краям, тип события |
| Выходные | `event_counts` (dict) | словарь «тип события → количество» |

### 7.3. Алгоритм / псевдокод

```
ФУНКЦИЯ count_security_events(log_lines)
  event_counts := пустой словарь
  ДЛЯ КАЖДОЙ line ИЗ log_lines
    clean_line := line без пробелов по краям
    ЕСЛИ clean_line пустая ТО следующая итерация        (continue)
    ЕСЛИ clean_line = "STOP" ТО выход из цикла           (break)
    event_type := часть clean_line до символа ";"
    ЕСЛИ event_type есть в event_counts ТО увеличить на 1
    ИНАЧЕ event_counts[event_type] := 1
  ВЕРНУТЬ event_counts
ВЫВОД пар «тип: количество»
```

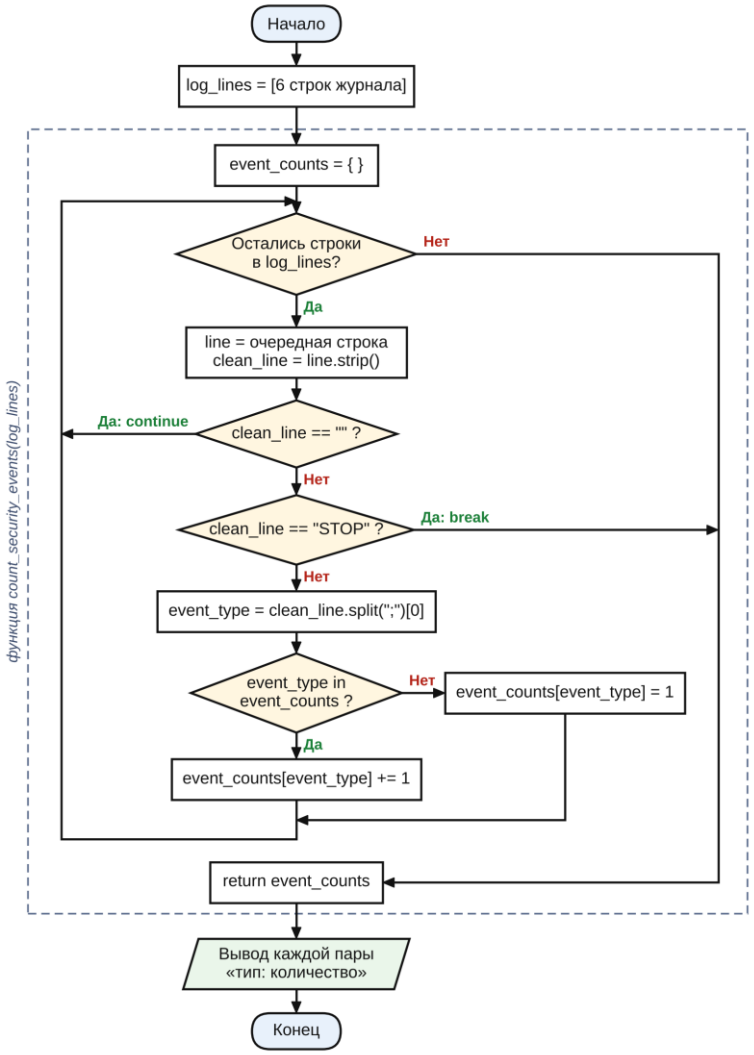



*Рисунок 5 — Блок-схема обработки строк журнала*

### 7.4. Программа на Python

```
def count_security_events(log_lines):
    event_counts = {}
    for line in log_lines:
        clean_line = line.strip()
        if clean_line == "":
            continue
        if clean_line == "STOP":
            break
        event_type = clean_line.split(";")[0]
        if event_type in event_counts:
            event_counts[event_type] += 1
        else:
            event_counts[event_type] = 1
    return event_counts


log_lines = [
    "login_failed;10.0.0.5",
    "",
    "login_success;192.168.1.10",
    "port_scan;172.16.10.25",
    "",
    "login_failed;10.0.0.8"
]

events = count_security_events(log_lines)
print("Результат анализа журнала:")
for event_type, amount in events.items():
    print(f"{event_type}: {amount}")
```

### 7.5. Тестирование и результат

| Входные данные | Ожидаемый результат | Фактический результат |
|---|---|---|
| список из задания | login_failed: 2; login_success: 1; port_scan: 1 | login_failed: 2; login_success: 1; port_scan: 1 |
| ["login_failed;10.0.0.1", "port_scan;10.0.0.2", "STOP", "login_failed;10.0.0.3"] | login_failed: 1; port_scan: 1 (после STOP не считается) | login_failed: 1; port_scan: 1 |
| ["", "  ", ""] — только пустые строки | пустой словарь {} | пустой словарь {} |

Вывод программы для данных из задания: login_failed: 2, login_success: 1, port_scan: 1 — совпадает с ожидаемым результатом методички.

### 7.6. Анализ

Пустые строки пропускаются, так как в них нет события: без `continue` выражение `"".split(";")[0]` дало бы ложный тип `""`. Строка `STOP` останавливает разбор через `break`. Словарь хранит для каждого типа события число его появлений. Для n строк временная сложность O(n), дополнительная память O(k), где k — число различных типов событий.

## 8. Общие выводы

В лабораторной работе применены условные конструкции `if/elif/else`, циклы `for` и `while`, функция `range()`, операторы `break` и `continue`, функции `def` с `return`. Для каждой задачи составлены алгоритм, псевдокод и блок-схема, выполнены трассировка и анализ сложности. Программы проверены запуском; результаты совпали с ожидаемыми (13 и 2, 408, login_failed: 2, login_success: 1, port_scan: 1).